C DATA TYPES

The Architecture of Information in C

Technical Encyclopedia | 18-Page Edition

01. Overview

In the C programming language, a **Data Type** is a classification that specifies which type of value a variable has and what type of mathematical, relational, or logical operations can be applied to it without causing an error.

> **Definition:** A data type specifies the type and size of data that a variable can store.

Data types are essential for memory management, efficiency, and the prevention of unexpected overflow or data loss.

02. Broad Categories

C data types are generally classified into four distinct categories:

1. **Basic (Primary) Types:** int, char, float, double.
2. **Derived Types:** Arrays, Pointers, Structures.
3. **Enumeration (User-Defined):** enum.
4. **Void Type:** Represents the absence of data.

This guide primarily focuses on **Basic** and **Extended** types.

03. The character type (char)

The `char` data type is used to store a single character, such as 'A', '7', or '#'.

```
char grade = 'A';
```

Key Points:

- Stored inside **single quotes** (' ').
- Size: **1 Byte** (8 bits).
- Range: -128 to 127 (or 0 to 255 for unsigned).
- Internally stored as **ASCII values**.

04. Numeric Types: Integers

Integers are whole numbers without fractional parts. In C, these are handled by the `int` family.

```
int count = 500;
```

Integers can be **Short** (2 bytes), **Standard** (4 bytes), or **Long** (8 bytes). They are signed by default, meaning they can hold both positive and negative values.

05. Numeric Types: Floating-Point

Used to represent numbers with decimal points. C provides three levels of precision:

- **float:** Single precision (4 bytes).
- **double:** Double precision (8 bytes).
- **long double:** Extended precision (10-16 bytes).

```
float price = 12.99f;
```

06. Understanding Precision

Precision represents how many digits after the decimal point are mathematically accurate.

Type	Accuracy	Approx. Range
float	~6-7 decimal digits	10^-38 to 10^38
double	~15-17 decimal digits	10^-308 to 10^308

In scientific computing, **double** is almost always preferred over **float**.

07. Memory Inventory

Memory sizes depend on system architecture (32-bit vs 64-bit), but standard sizes are:

Data Type	Standard Size
char	1 Byte
int	4 Bytes
float	4 Bytes
double	8 Bytes

08. Dynamic Size Checking

Since data type sizes can change based on the machine, C provides the `sizeof()` operator to check the size at runtime.

```
printf("Int: %lu bytes", sizeof(int));
printf("Long: %lu bytes", sizeof(long));
```

This is crucial for writing **portable code** that runs on different processors.

09. Communication via Specifiers

The compiler needs to know how to interpret binary data. We communicate this using format specifiers in `printf` and `scanf`.

- **%d** : Signed Integer
- **%c** : Character
- **%f** : Float
- **%lf** : Double
- **%u** : Unsigned Integer

10. Mapping Reality to Code

Choosing the correct type is a design skill:

- **Student Age:** `int`
- **Pi Constant:** `double`
- **Blood Group:** `char`
- **GPS Coordinates:** `long double`
- **Bank Balance:** `double`

11. Extended Variations

C allows modifiers like `signed`, `unsigned`, `short`, and `long`.

```
unsigned int positiveOnly = 100;
long long int nationalDebt = 9999999999;
```

The `unsigned` modifier doubles the positive range by removing negative capability.

12. Characters as Integers

In C, a `char` is actually a small integer. You can perform math on characters!

```
char result = 'A' + 1; // result is 'B'
```

This works because 'A' is mapped to 65 in the ASCII table, and 65 + 1 = 66 ('B').

13. Technical Pitfalls

- **Overflow:** Trying to store 50,000 in a 2-byte short (max 32,767).
- **Truncation:** Storing 3.99 in an `int` variable; the .99 is lost (result is 3).
- **Format Mismatch:** Using %d to print a float causes garbage output.

14. Implicit vs Explicit Casting

Implicit: Done by compiler (int to float).

Explicit: Done by programmer to avoid data loss.

```
(float)5 / 2; // Returns 2.5 instead of 2
```

15. The Lab: Hands-on Challenge

Predict the output of the following block:

```
int a = 10;
float b = 2.5;
printf("%.2f", a * b);
```

Challenge: Can you identify the size of the result in memory?

16. Beyond Basic Types

As programs grow, basic types are insufficient. C allows you to bundle data:

- **Arrays:** Collection of same data types.
- **Structures (struct):** Collection of different types.
- **Typedef:** Creating aliases for types.

17. Final Summary

- **char** (1 byte, %c)
- **int** (4 bytes, %d)
- **float** (4 bytes, %f)
- **double** (8 bytes, %lf)

Mastering data types is the first step toward memory-optimized programming.

END OF DATA TYPES GUIDE